Data Filtering, Aggregation, and Summarisation in Spark

In this train, we will explore how to filter, aggregate, and summarize data using Apache Spark, deriving meaningful insights through Spark's robust framework.

Learning objectives

By the end of this train you should be able to:

- [Placeholder for objective 1]
- [Placeholder for objective 2]
- [Placeholder for objective 3]

Learning Objectives

- Filter, aggregate, and summarize data using Spark.

Filtering, Aggregation, and Summarisation

Part of the process of making data more manageable and consistent is filtering, aggregating, and summarizing the data. This can be done by filtering out null values, invalid records, or irrelevant fields. A large part of this phase is the exploration of the dataset for such errors and challenges, a process that is analogous to a data scientist performing an Exploratory Data Analysis.

It can also be done by aggregating data to a level that is more appropriate for analytical teams, or which will simplify the visualization of datasets. For example, aggregating data up to a high geographical level (for visualizing COVID-19 cases at the level of a country vs. a city) or aggregating time-series data up to the hour instead of the second.

There is an almost inexhaustible list of errors that can be present within your dataset when performing these tasks. We will try to cover some of the most prominent issues that you may encounter:

duplicated data;

spurious observations;

structural errors; and

missing observations.

We will deal with the first three in this section and take a look at the fourth but only deal with it in the next section.

Now that we have the data in parquet format, we can easily read it in again without having to specify the schema again.

```
# Read the data in using the read.parquet() method from existing parquet files.

working_df = spark.read.parquet('./data/weather_data/hrly_Irish_weather/')
```

Make a list of categorical and continuous variables. This may be useful if we want to filter by these variable fields later.

```
weather_cat = ['county', 'station', 'date']
weather_int = ['rhum', 'wdsp', 'wddir', 'vis', 'clht', 'clamt']

# Create a list of continuous fields (fields that are not in the categorical
list above).
weather_cont = [x for x in working_df.columns if x not in weather_cat]

# Create a list of float fields (those in the above list, but not in the integer
list).
weather_float = [x for x in weather_cont if x not in weather_int]
```

1. Duplicated Data

A feature of uncleansed data may sometimes be the occurrence of duplicated data points. Removing these redundant pieces of data forms part of the filtering process. The origin of the duplicates may be the result of an internal process (true duplicates) or due to a failure in the system, such as incorrect data capture or incorrect data processing. To combat such duplications, it is always important to check and remove duplicate entries within a dataset, and this is done through a deduplication process. Deduplication can be simple and is normally based on numerical entries or entries that are well indexed. An example of this might be transactional data in a point-of-sale system, which has a primary key but was processed incorrectly. Table 1. An example of duplicated records.

Index	Date	Name	Value
1	2021-01-01	Ben	20
2	2021-01-01	Sarah	12
3	2021-01-01	Jack	24
3	2021-01-01	Jack	24
4	2021-01-01	Ben	20

In the above table, we demonstrate the types of duplications that can occur. Rows with index 3 indicate two duplicates that likely resulted from a system failure, causing the entry to be duplicated within our system. We should remove this type of duplicate from our system. Rows with index 1 and 4 are true duplicates according to the above definition and are likely a manifestation of the underlying process. Ben probably went to the shop twice on the 1st of January 2021 and bought goods to the value of 20. We would want to keep these duplicates in our system as they likely represent real processes. Alternatively, deduplication can be very complex, with the only unique identifier being a string column, in which case, fuzzy logic or some similar technique has to be applied to identify and remove duplicates. An example of this might be names that are in an HR list, some of which have been entered incorrectly, or consolidation between names in multiple systems. Let's start by checking if all entries within the datasets are unique.

```
working_df.groupby(working_df.columns) \
  .count() \
  .where('count > 1') \
  .sort('count', ascending=False) \
  .show()
```

Output:

```
root
 |-- county: string (nullable = true)
 |-- station: string (nullable = true)
 |-- latitude: float (nullable = true)
 |-- longitude: float (nullable = true)
 |-- date: timestamp (nullable = true)
 |-- rain: float (nullable = true)
 |-- temp: float (nullable = true)
 |-- wetb: float (nullable = true)
 |-- dewpt: float (nullable = true)
 |-- vappr: float (nullable = true)
 |-- rhum: integer (nullable = true)
 |-- msl: float (nullable = true)
 |-- wdsp: integer (nullable = true)
 |-- wddir: integer (nullable = true)
 |-- sun: float (nullable = true)
 |-- vis: integer (nullable = true)
 |-- clht: integer (nullable = true)
 |-- clamt: integer (nullable = true)
 |-- day: integer (nullable = true)
 |-- month: integer (nullable = true)
 |-- season: string (nullable = true)
 |-- idx: long (nullable = false)
```

Here we are using the `groupby()` method with all the columns as argument, meaning we are grouping by all the columns. We then use the `count()` aggregation method, modify that with the `where()` method, and only retrieving rows where the count is larger than one. Finally, we sort by the count column using the `sort()` method and display the resulting DataFrame. That looks about right... no duplicates across the whole dataset! It does, however, feel like we expected that. It is also just more plausible, statistically. What if we try to create a compound primary index using the date and the station? That should still be unique across the dataset. We expect there to be no double entry per station at any one point in time.

```
working_df.groupby(['date', 'station']) \
  .count() \
  .where('count > 1') \
  .sort('count', ascending=False) \
  .show()
```

Output:

```
root
 |-- County: string (nullable = true)
 |-- Weather Station: string (nullable = true)
 |-- Latitude: float (nullable = true)
 |-- Longitude: float (nullable = true)
 |-- Date: string (nullable = true)
 |-- Rain (mm): float (nullable = true)
 |-- Temperature (°C): float (nullable = true)
 |-- Wet Bulb Air Temperature (°C): float (nullable = true)
 |-- Dew Point: float (nullable = true)
 |-- Vapour Pressure (hPa): float (nullable = true)
 |-- Relative Humidity (%): integer (nullable = true)
 |-- Mean Sea Level Pressure (hPa): float (nullable = true)
 |-- Mean Hourly Wind Speed (kt): integer (nullable = true)
 |-- Predominant Hourly Wind Direction (degrees): integer (nullable = true)
 |-- Sun (hours): float (nullable = true)
 |-- Visability (m): integer (nullable = true)
 |-- Cloud Ceiling Height: integer (nullable = true)
 |-- Cloud Ammount (Oktas): integer (nullable = true)
 |-- Day: integer (nullable = true)
 |-- Month: integer (nullable = true)
 |-- Season: string (nullable = true)
 |-- Index: long (nullable = false)
```

Here we are doing exactly the same as above, except we're only grouping by the date and station columns. No duplicates... nice!

2. Spurious Observations

Another feature of a dataset that has not yet gone through data cleansing may be the occurrence of spurious observations. Spurious observations may have many origins: humans performing data capture may enter incorrect data, an incorrect understanding of the input data may lead to incorrect entries (for example, 0 being entered as O), or automated data capture processes may lead to incorrect entries in a similar way (for example, an OCR system may read an S in as a 5). Other errors may be even more domain-specific (for example, an acoustic sewer level sensor may give negative readings when the water level is above the sensor, or when covered by a large object). It is clear that we have to make sure that all the values within the input data

make sense within the bounded context of the domain we are working in. For our dataset, we have defined the field identities above. Let's revisit them and define possible bounds for them:

rain – Rain (mm) – any positive decimal number.

temp – Temperature (°C) – any decimal number above -273.15 (expect to be between -20 and 40).

wetb – Wet Bulb Air Temperature (°C) – any decimal number above -273.15 (expect to be between -20 and 40).

dewpt – Dew Point – any decimal number above -273.15 (expect to be between -20 and 40).

vappr – Vapour Pressure (hPa) – any positive decimal number.

rhum – Relative Humidity (%) – between 0 and 100.

msl – Mean Sea Level Pressure (hPa) – any positive decimal number (expect close to 1,013.25).

wdsp – Mean Hourly Wind Speed (kt) – any positive decimal number (maximum at 220 - fastest ever recorded).

wddir – Predominant Hourly Wind Direction (degrees) – between 0 and 360.

sun – Sun (hours) – between 0 and 24.

vis – Visibility (m) – any positive decimal number.

clht – Cloud Ceiling Height – between 0 and 999.

clamt – Cloud Amount (Oktas) – between 0 and 9.

Let's quickly run through checks for all of the above:

1. Rain > 0

Rain should be positive. Let's see if we have any negative records:

```
working_df.where(F.col('rain') < 0).show()
```

Output:

```
+------+-------+--------+---------+----+----+----+-----+-----+-----+----+---+----+-----+---+---+----+-----+
|county|station|latitude|longitude|date|rain|temp|wetb|dewpt|vappr|rhum|msl|wdsp|wddir|sun|vis|clht|clamt|
+------+-------+--------+---------+----+----+----+-----+-----+-----+----+---+----+-----+---+---+----+-----+
+------+-------+--------+---------+----+----+----+-----+-----+-----+----+---+----+-----+---+---+----+-----+
```

2. Temp Between -20 and 40

```
working_df.where(~F.col('temp').between(-20, 40)).show()
```

Output:

```
+------+-------+--------+---------+----+----+----+----+-----+-----+----+---+----+-----+---+---+----+-----+
|county|station|latitude|longitude|date|rain|temp|wetb|dewpt|vappr|rhum|msl|wdsp|wddir|sun|vis|clht|clamt|
+------+-------+--------+---------+----+----+----+----+-----+-----+----+---+----+-----+---+---+----+-----+
+------+-------+--------+---------+----+----+----+----+-----+-----+----+---+----+-----+---+---+----+-----+
```

But... I thought we are filtering records 'where the temp is between -20 and 40'. Why are we not getting anything? This is where the `~` operator comes in, which means 'everything except'.

3. Wetb Between -20 and 40

```
working_df.where(~F.col('wetb').between(-20, 40)).show(10)
```

Output:

```
+------+--------+--------+---------+-------------------+----+----+-----+-----+-----+----+------+----+-----+---+---
--+----+-----+
|county| station|latitude|longitude|               date|rain|temp| wetb|dewpt|vappr|rhum|   msl|wdsp|wddir|sun|  v
is|clht|clamt|
+------+--------+--------+---------+-------------------+----+----+-----+-----+-----+----+------+----+-----+---+---
--+----+-----+
|Dublin|CASEMENT|  53.306|   -6.439|2017-08-01 00:00:00| 0.0|12.5|-49.0| 11.1| 13.3|  92|1009.2|   6|  210|0.0|200
00|  70|    7|
|Dublin|CASEMENT|  53.306|   -6.439|2017-08-31 21:00:00| 0.0| 8.3|-49.0|  7.0| 10.0|  92|1021.4|   4|  210|0.0|300
00| 999|    2|
|Dublin|CASEMENT|  53.306|   -6.439|2017-09-01 00:00:00| 0.0| 8.0|-49.0|  6.8|  9.8|  92|1022.5|   4|  220|0.0|200
00| 999|    1|
|Dublin|CASEMENT|  53.306|   -6.439|2018-01-28 10:00:00| 0.0|11.7|-49.0| 10.3| 12.6|  91|1025.5|  20|  240|0.0| 90
00|  11|    8|
|Dublin|CASEMENT|  53.306|   -6.439|2019-06-30 08:00:00| 0.0|16.1|-49.0| 12.6| 14.7|  80|1012.5|  14|  230|0.9|400
00| 999|    2|
|Dublin|CASEMENT|  53.306|   -6.439|2019-06-30 09:00:00| 0.0|16.8|-49.0| 11.7| 13.7|  72|1012.6|  15|  230|0.1|400
00|  50|    6|
|Dublin|CASEMENT|  53.306|   -6.439|2019-06-30 10:00:00| 0.0|16.7|-49.0| 11.6| 13.6|  72|1013.0|  15|  220|0.0|300
00|  30|    6|
|Dublin|CASEMENT|  53.306|   -6.439|2019-06-30 11:00:00| 0.0|16.9|-49.0| 12.6| 14.6|  76|1013.2|  15|  230|0.1|300
00|  30|    6|
|Dublin|CASEMENT|  53.306|   -6.439|2019-06-30 12:00:00| 0.6|15.4|-49.0| 14.7| 16.7|  96|1013.7|  13|  250|0.3|250
00|  40|    6|
|Dublin|CASEMENT|  53.306|   -6.439|2019-06-30 13:00:00| 0.4|16.6|-49.0| 13.9| 16.0|  85|1013.5|  14|  240|0.7|250
00|  50|    6|
+------+--------+--------+---------+-------------------+----+----+-----+-----+-----+----+------+----+-----+---+---
--+----+-----+
only showing top 10 rows
```

Weird... let's have a look at how many of these we have and how extreme they are:

```
working_df.where(~F.col('wetb').between(-20, 40)).groupBy('wetb',
'station').count().show()
```

Output:

```
+-----+---------------+-----+
| wetb|        station|count|
+-----+---------------+-----+
|-49.0|   ROCHES POINT|   99|
|-49.0|   CORK AIRPORT|   92|
|-99.9|    CLAREMORRIS|    9|
|-49.0|       CASEMENT|   86|
|-49.0|SHANNON AIRPORT|  103|
|-49.9| DUBLIN AIRPORT|    1|
+-----+---------------+-----+
```

That is quite a lot. It is also only three very specific values spread across many stations. Let's look at where they are located to see if we can find a pattern:



Figure 1: A map of Ireland.

They are not at all clustered or far north. We expect these to be default values that the loggers return if they are faulty. We can note this and handoff to the modeling teams or remove these values.

4. Dewpt Between -20 and 40

```
working_df.where(~F.col('dewpt').between(-20, 40)).show(10)
```

Output:

```
+------+-----------+--------+---------+-------------------+----+----+----+-----+-----+----+------+----+-----+----+
----+----+-----+
|county|    station|latitude|longitude|               date|rain|temp|wetb|dewpt|vappr|rhum|   msl|wdsp|wddir| sun|
vis|clht|clamt|
+------+-----------+--------+---------+-------------------+----+----+----+-----+-----+----+------+----+-----+----+
----+----+-----+
|  Mayo|CLAREMORRIS|  53.711|   -8.993|2003-01-30 18:00:00| 0.0| 0.4|-4.0|-22.3|  1.0|  16|1030.6|   8|   10|null|
null|null| null|
|  Mayo|CLAREMORRIS|  53.711|   -8.993|2003-01-30 19:00:00| 0.0|-0.2|-4.5|-22.9|  0.9|  16|1030.9|   5|  310|null|
null|null| null|
|  Mayo|CLAREMORRIS|  53.711|   -8.993|2003-01-30 20:00:00| 0.0| 0.4|-4.0|-22.3|  1.0|  16|1031.3|   6|  320|null|
null|null| null|
|  Mayo|CLAREMORRIS|  53.711|   -8.993|2003-01-30 21:00:00| 0.0|-0.2|-4.5|-22.8|  0.9|  16|1031.7|   5|  310|null|
null|null| null|
|  Mayo|CLAREMORRIS|  53.711|   -8.993|2003-01-30 22:00:00| 0.0| 0.1|-4.2|-22.1|  1.0|  17|1031.5|   5|  300|null|
null|null| null|
|  Mayo|CLAREMORRIS|  53.711|   -8.993|2003-01-30 23:00:00| 0.0|-0.6|-4.7|-22.7|  1.0|  18|1031.6|   4|  300|null|
null|null| null|
|  Mayo|CLAREMORRIS|  53.711|   -8.993|2003-01-31 00:00:00| 0.0| 0.0|-4.3|-22.2|  1.0|  17|1031.9|   5|  270|null|
null|null| null|
|  Mayo|CLAREMORRIS|  53.711|   -8.993|2003-01-31 01:00:00| 0.0| 0.4|-4.0|-21.8|  1.0|  16|1031.8|   5|  290|null|
null|null| null|
|  Mayo|CLAREMORRIS|  53.711|   -8.993|2003-01-31 02:00:00| 0.0| 0.5|-3.9|-21.7|  1.1|  17|1031.2|   3|  300|null|
null|null| null|
|  Mayo|CLAREMORRIS|  53.711|   -8.993|2003-01-31 03:00:00| 0.0| 0.3|-4.0|-21.9|  1.1|  18|1030.5|   3|  270|null|
null|null| null|
+------+-----------+--------+---------+-------------------+----+----+----+-----+-----+----+------+----+-----+----+
----+----+-----+
only showing top 10 rows
```

```
working_df.where(~F.col('dewpt').between(-20, 40)).groupBy('dewpt',
'station').count().show()
```

Output

```
+-----+-----------+-----+
|dewpt|    station|count|
+-----+-----------+-----+
|-21.9|CLAREMORRIS|    4|
|-22.9|CLAREMORRIS|    1|
|-21.7|CLAREMORRIS|    2|
|-22.1|CLAREMORRIS|    1|
|-21.8|CLAREMORRIS|    2|
|-22.8|CLAREMORRIS|    1|
|-22.7|CLAREMORRIS|    1|
|-22.2|CLAREMORRIS|    1|
|-22.3|CLAREMORRIS|    2|
|-21.3|CLAREMORRIS|    1|
|-92.4| MOORE PARK|   15|
+-----+-----------+-----+
```

We could have been wrong in our assumption of no readings lower than -20 since the readings at CLAREMORRIS seem to be legitimate readings. However, the MOORE PARK reading seems too low, and as above with `wetb`, we should remove these values. Those were all relative examples, for example, checking if temperatures fell within a specific range. Let's check for dependencies between columns. We would expect the dew point to be lower than the current temperature. Let's see if that is the case:

```
working_df.where(F.col('dewpt') > F.col('temp')).show(10)
```

Output:

```
+------+---------+--------+---------+-------------------+----+----+----+-----+-----+----+------+----+-----+----+-
---+----+-----+
|county|   station|latitude|longitude|               date|rain|temp|wetb|dewpt|vappr|rhum|   msl|wdsp|wddir| sun|
vis|clht|clamt|
+------+---------+--------+---------+-------------------+----+----+----+-----+-----+----+------+----+-----+----+-
---+----+-----+
|Galway|   ATHENRY|  53.289|   -8.786|2012-12-10 06:00:00| 0.0|-3.0|-2.9| -2.9|  4.9| 100|1025.9|   2|  140|null|n
ull|null| null|
|Galway|   ATHENRY|  53.289|   -8.786|2013-01-09 09:00:00| 0.0|-1.4|-1.3| -1.3|  5.5| 100|1022.9|   3|   90|null|n
ull|null| null|
|Galway|   ATHENRY|  53.289|   -8.786|2013-01-15 08:00:00| 0.0|-1.7|-1.6| -1.6|  5.4| 100|1014.7|   1|  200|null|n
ull|null| null|
|Galway|   ATHENRY|  53.289|   -8.786|2013-03-17 06:00:00| 0.0|-2.9|-2.8| -2.8|  5.0| 100| 991.4|   1|   40|null|n
ull|null| null|
|Galway|   ATHENRY|  53.289|   -8.786|2013-03-28 05:00:00| 0.0|-1.8|-1.7| -1.7|  5.4| 100|1015.2|   6|   50|null|n
ull|null| null|
| Cavan|BALLYHAISE|  54.051|    -7.31|2007-10-25 08:00:00| 0.0|-1.8|-1.7| -1.7|  5.4| 100|1026.0|   3|  140|null|n
ull|null| null|
| Cavan|BALLYHAISE|  54.051|    -7.31|2008-02-17 13:00:00| 0.0|-0.1|-0.0| -0.0|  6.1| 100|1039.8|   1|  100|null|n
ull|null| null|
| Cavan|BALLYHAISE|  54.051|    -7.31|2011-11-06 04:00:00| 0.0|-1.2|-1.1| -1.1|  5.6| 100|1024.2|   3|  100|null|n
ull|null| null|
| Cavan|BALLYHAISE|  54.051|    -7.31|2014-03-12 08:00:00| 0.0|-2.1|-2.0| -2.0|  5.3| 100|1033.4|   1|  100|null|n
ull|null| null|
|Dublin|  CASEMENT|  53.306|   -6.439|2008-02-19 00:00:00| 0.0|-0.1|-0.1|  0.0|  6.1| 100|1028.4|   2|  140| 0.0|5
000| 999|    0|
+------+---------+--------+---------+-------------------+----+----+----+-----+-----+----+------+----+-----+----+-
---+----+-----+
only showing top 10 rows
```

```
working_df.where(F.col('dewpt') >
F.col('temp')).groupBy('county').count().show()
```

Output:

```
+---------+-----+
|   county|count|
+---------+-----+
|    Clare|    1|
|Roscommon|   31|
|   Dublin|  103|
|   Galway|    5|
|     Cork|  106|
|Tipperary|   10|
|     Mayo|   54|
|    Meath|    7|
|   Carlow|   24|
|Westmeath|   20|
|    Sligo|    2|
|    Kerry|    2|
|    Cavan|    4|
+---------+-----+
```

You can see that it's possible to do comparisons between columns as well. It does not look like we can use the relationship between temperature and dew point to determine valid entries. If we look at the counties where there are dew points higher than the temperatures, there are quite a few.

> **[Exercise]**, complete the data checks for the remainder of the expected values:
>
> - 5. vappr a positive decimal
> - 6. rhum between 0 and 100
> - 7. msl any positive decimal number close to 1013.25
> - 8 wdsp any positive decimal number below 220
> - 9. wddir between 0 and 360

- 10. sun between 0 and 24
- 11. vis any positive decimal number
- 12. clht integer between 0 and 999
- 13. clamt integer between 0 and 9

List of errors:

- 1. wetb removes values where wetb == 49.0, 49.9 and 99.0
- 2. dewpt removes values where dewpt == 92.4
- 3. rhum removes values where rhum == -14
- 4. clamt removes null values in the clamt field

Conclusion

In this lesson, we explored how to filter, aggregate, and summarize data using Spark. We demonstrated how to detect and handle duplicated and spurious records, ensuring cleaner and more reliable datasets for analysis.

▼Key Takeaways ✖

- **Takeaway 1:** Use Spark to filter, aggregate, and summarize large datasets efficiently.
- **Takeaway 2:** Remove duplicates by grouping data and checking for repeated records.
- **Takeaway 3:** Detect spurious values by applying domain-specific rules and value ranges.
- **Takeaway 4:** Clean data improves accuracy and quality of insights in downstream analytics.